Voice-Based Parkinson's Disease Classification

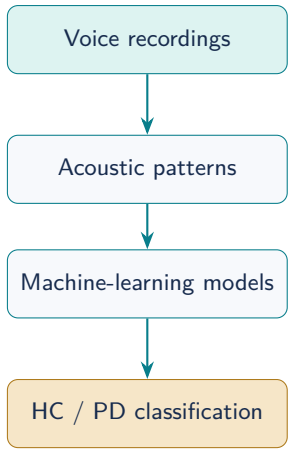
Handcrafted Acoustic Features and Log-Mel Spectrograms

Suvinay Foujdar Akshat Goyal Navya Loshali

Department of Computer Science and Engineering
Greater Noida, India

Why analyse voice for Parkinson's disease?

- Parkinson's disease can affect phonation, articulation, timing and prosody.
- Voice recordings offer a non-invasive source of acoustic information.
- Repeated recordings from the same speaker are correlated.
- A random recording-level split can leak speaker-specific information into testing.



Research question

How do handcrafted acoustic summaries and a Log-Mel CNN compare when evaluation is performed on participants excluded from model development?

Objectives and study contribution

Objectives

1. Audit the dataset and participant-folder structure.
2. Build a handcrafted acoustic-feature pipeline.
3. Train and compare four classical classifiers.
4. Build a CNN using five-second Log-Mel inputs.
5. Evaluate both approaches on the same held-out participants.

What this project contributes

- Participant-disjoint evaluation.
- Recording-level and participant-level reporting.
- Side-by-side classical ML and CNN implementation.
- Exploratory permutation-importance analysis.

Implementation-focused comparison; not a new clinical biomarker.

Dataset: Italian Parkinson's Voice and Speech (IPVS)

65
participants

831
WAV recordings

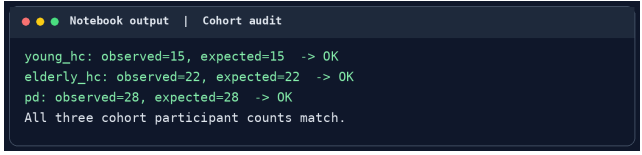
115
handcrafted feature columns per recording

Cohort	People	Recordings
Young healthy controls	15	45
Elderly healthy controls	22	349
Parkinson's disease	28	437
Total	65	831

Important cohort detail

The two healthy-control cohorts are combined for binary HC/PD classification, but their original identities are retained during the audit.

Dataset audit: verified by the notebook

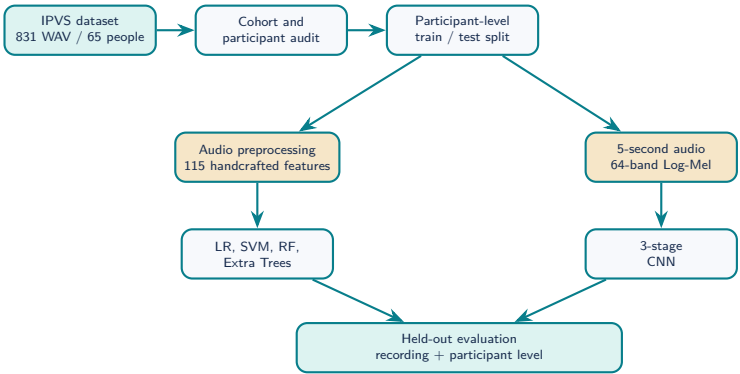


- Cohort counts matched expected folder labels.
- 65 unique participants identified.
- 831 feature rows created.
- 0 feature-extraction errors in the saved run.

Why this matters

The audit checks the real participant folders before any model is trained. It prevents range folders in the PD directory from being mistaken for participants.

End-to-end methodology



Leakage control: All recordings from one participant stay in a single partition throughout development and final testing.

Audio preprocessing and handcrafted features

1. Load WAV and convert stereo to mono.
2. Resample to 16 kHz.
3. Mean-centre and peak-normalise.
4. Extract frame-level acoustic descriptors.
5. Summarise each sequence using mean and standard deviation.

Feature groups

20 MFCC + delta-MFCC, spectral centroid, bandwidth, rolloff, zero-crossing rate, RMS, chroma and duration.

Notebook cell 16 | Handcrafted feature extraction

```python
def extract_features(path):
    y, sr = sf.read(path, dtype='float32')
    if y.ndim == 2:
        y = y.mean(axis=1)
    if y.size == 0:
        raise ValueError('Empty audio')
    if sr != TARGET_SR:
        y = librosa.resample(y, orig_sr=sr, target_sr=TARGET_SR)
        sr = TARGET_SR
    y = y - np.mean(y)
    peak = np.max(np.abs(y))
    if peak > 0:
        y = y / (peak + 1e-9)

    out = {'duration_sec': float(len(y) / sr)}
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=N_MFCC)
    delta = librosa.feature.delta(mfcc, mode='nearest')
    mean_std_features(mfcc, 'mfcc', out)
    mean_std_features(delta, 'delta_mfcc', out)

    for name, arr in {
        'spectral_centroid': librosa.feature.spectral_centroid(y=y, sr=sr),
        'spectral_bandwidth': librosa.feature.spectral_bandwidth(y=y, sr=sr),
```

Code screenshot: actual feature-extraction function from notebook cell 16 (excerpt).

Participant-disjoint split: the key design decision

Notebook cell 20 | Participant-level split

```python
# Participant-level stratified train/test split (no participant leakage)

X = features_df[feature_cols].copy()
y = features_df['label'].astype(int).copy()
groups = features_df['participant'].astype(str).copy()

# One row per participant is used to stratify the split.
participant_table_all = (
    features_df[['participant', 'label']]
    .drop_duplicates()
    .reset_index(drop=True)
)

assert participant_table_all.groupby('participant')['label'].nunique().max() == 1

participant_splitter = StratifiedShuffleSplit(
    n_splits=1, test_size=0.25, random_state=SEED
)

participant_train_pos, participant_test_pos = next(
    participant_splitter.split(
        participant_table_all['participant'],
        participant_table_all['label']
    )
)
```

Notebook output | Participant-disjoint split

```
Participant-wise stratified split successful
Train recordings: 605  |  Test recordings: 226
Train participants: 48  |  Test participants: 17
Train participants: HC 27  |  PD 21
Test participants:  HC 10  |  PD 7
```

Saved split

48 development people
27 HC + 21 PD

17 held-out people
10 HC + 7 PD

Code and output: actual notebook cell 20 and its saved execution output.

Classical machine-learning comparison

- Logistic Regression
- RBF-kernel Support Vector Machine
- Random Forest
- Extra Trees

Model selection used five-fold participant-stratified cross-validation on the 48 development participants only.

Model	Mean BA	SD
RBF-SVM	**0.9606**	0.0360
Extra Trees	0.9483	0.0474
Random Forest	0.9404	0.0469
Logistic Reg.	0.9328	0.0302

Selected model

RBF-SVM — highest mean balanced accuracy in development cross-validation.

CNN: learning from Log-Mel spectrograms

Notebook cell 32 | Log-Mel CNN architecture

```
# CNN training with participant-aware sample weights and group-wise validation

cnn = tf.keras.Sequential([
    tf.keras.layers.Input(shape=X_cnn_train.shape[1:]),
    tf.keras.layers.Conv2D(16, 3, padding='same', activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, padding='same', activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, padding='same', activation='relu'),
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.45),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.35),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

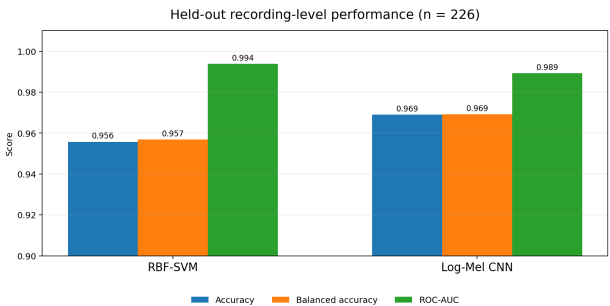
cnn.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=3e-4),
    loss='binary_crossentropy',
    metrics=[
```

- Five-second crop from the beginning of each recording.
- 64 Mel bands; FFT 1024; hop length 256.
- Input tensor: $64 \times 313 \times 1$.
- Conv2D filters: 16, 32, 64.
- Batch normalisation, max pooling, global average pooling.
- Dropout 0.45 / 0.35; Dense 32; sigmoid output.

Training

Adam (3×10^{-4}), binary cross-entropy, batch size 16, up to 60 epochs, early stopping.

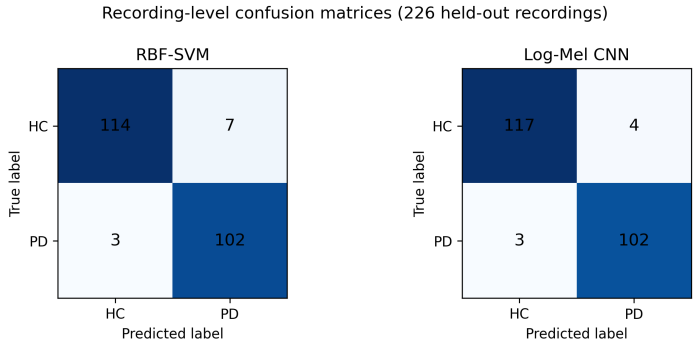
Held-out test results: both levels matter



	SVM	CNN
Recording accuracy	.9558	**.9690**
Recording BA	.9568	**.9692**
PD precision	.9358	**.9623**
PD recall	.9714	.9714
PD F1	.9533	**.9668**
ROC-AUC	**.9939**	.9892
Participant accuracy	.9412	.9412
Participant BA	.9500	.9500

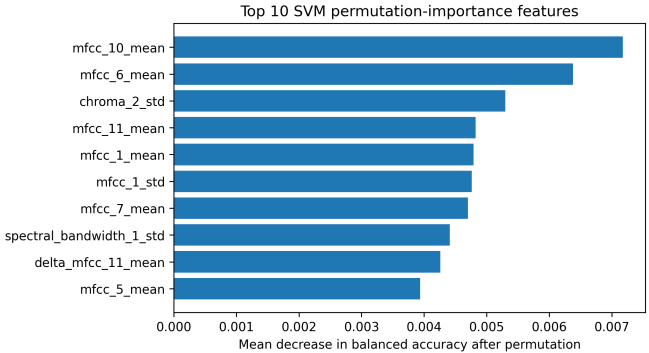
Test cohort: 226 recordings from 17 held-out participants. Participant-level metrics are identical in this saved run.

What did the models get right and wrong?



SVM: 114 HC and 102 PD correct; 7 HC and 3 PD misclassified.
CNN: 117 HC and 102 PD correct; 4 HC and 3 PD misclassified.

Exploratory feature importance



- MFCC summary variables appear frequently among top-ranked features.
- Chroma variability and spectral bandwidth also contribute.
- Importance is model- and dataset-dependent.

Interpret carefully

Permutation importance is exploratory; it does not prove causality or identify a clinically validated biomarker.

Limitations and future work

Limitations

- One dataset and one saved participant split.
- Only 17 independent held-out participants.
- HC combines young and elderly cohorts; age effects are not isolated.
- CNN uses only the first five seconds.
- No external or prospective clinical validation.

Next steps

- Validate on independent datasets and more participants.
- Repeat grouped outer splits / nested grouped validation.
- Test segment-selection strategies and recording robustness.
- Assess calibration and uncertainty.
- Evaluate prospectively with clinical oversight.

Conclusion

- Implemented two participant-aware pipelines for voice-based HC/PD classification.
- Handcrafted branch: 115 acoustic summary features; RBF-SVM selected by grouped CV.
- Deep-learning branch: five-second, 64-band Log-Mel spectrograms processed by a compact CNN.
- Recording-level accuracy: 95.58% (SVM) and 96.90% (CNN).
- Participant-level accuracy: 94.12% for both models on 17 held-out people.

Participant-aware evaluation shows strong discrimination on the held-out cohort.

Further validation on independent participants and datasets is the next step.

Thank you.
Questions?

Selected references

I. Luna-Ortiz et al., "Parkinson's Disease Detection from Voice Recordings Using Associative Memories," *Healthcare*, vol. 11, no. 11, 1601, 2023.

Q. C. Ngo et al., "Computerized analysis of speech and voice for Parkinson's disease: A systematic review," *Comput. Methods Programs Biomed.*, vol. 226, 107133, 2022.

F. Amato et al., "Machine learning- and statistical-based voice analysis of Parkinson's disease patients: A survey," *Expert Syst. Appl.*, vol. 219, 119651, 2023.

D. Xavier et al., "Voice analysis in Parkinson's disease—a systematic literature review," *Artif. Intell. Med.*, vol. 163, 103109, 2025.

H. Sedigh Malekroodi, B.-I. Lee, and M. Yi, "Voice-Based Detection of Parkinson's Disease Using Machine and Deep Learning Approaches: A Systematic Review," *Bioengineering*, vol. 12, no. 11, 1279, 2025.

M. A. Hossain, E. Traini, and F. Amenta, "Identification of Parkinson's Disease from Native Italian People: Machine Learning Voice Analysis," *BioMed*, vol. 6, no. 3, 15, 2026.